

Epic / Private key storage and cross-device account access ▾

In theory would signal crypto and everything work if I made an account on the cpp client and then tried accessing that account on a browser application on a new device? Would I still have functionality? Is my private key stored locally or smtjn?

Searched memory >

Good question — this gets at a fundamental design decision in your system. Here's the honest answer:

Short answer: No, not out of the box — and that's by design (and also true of Signal itself).

Where private keys live, per client:

C++ client: The X25519 + Ed25519 identity keypair is stored in an encrypted key file on disk (`~/.securemsg/` or `$SECUREMSG_HOME`), protected by Argon2id + AES-256-GCM. The private keys *never* leave that machine — `sodium_malloc` keeps them in locked, non-swappable memory at runtime.

Browser client: Private keys are stored as *non-extractable* `CryptoKey` objects in IndexedDB. They are literally unextractable by design — the Web Crypto API prevents you from ever reading the raw bytes out. They never hit the server.

Python daemon: Session state (ratchet root key, chain keys, etc.) is encrypted with AES-256-GCM and persisted to `~/.securemsg/sessions/`.

So what happens if you try to log into a second device?

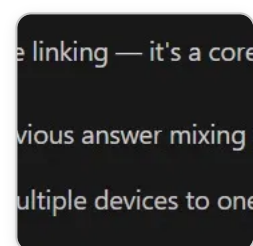
You'd be logging in with the same credentials, but the new device has **no private keys and no ratchet session state**. Specifically:

- The new device can't decrypt any existing messages (message keys are ephemeral and derived locally — the server only stores ciphertext)
- The new device would need to generate a *new* identity keypair, which would overwrite your key bundle on the server — breaking sessions for anyone who had already done X3DH with your old IK

- Any Double Ratchet sessions are entirely local state — not synced to the server (by design, and for good security reasons)

This is intentional and matches Signal's model. Signal doesn't let you seamlessly share an account across devices either — multi-device in Signal is a whole separate protocol (Sealed Sender + linked devices). Your EPIC system has the same fundamental constraint.

For your demo/submission purposes: each account is effectively tied to the device it was created on. That's actually a *feature* you can articulate in your crypto design doc — the server never sees private key material, which is the whole point.



Is this gobble Dee goo

That's from another Claude conversation, and it's... not accurate. The previous response I gave you was correct.

Signal does support device linking, yes — but **EPIC does not implement this**, and that's completely fine for your project. Device linking in Signal is a complex additional protocol on top of the core crypto. What I said stands:

- Your private keys are **device-local only** (locked in `sodium_malloc` on C++, non-extractable IndexedDB on browser)
- Ratchet session state is **local only**
- A second device logging into the same account would have no keys and couldn't decrypt anything

The screenshot is wrong to say "EPIC does this ✓" — EPIC implements the core Signal crypto (X3DH + Double Ratchet), not device linking. Those are separate things. Don't let that other response confuse you.

okay so rn its one account per instance of the client - so you can have one account on cpp and one account on browser but thye must be seperate?

Yes, exactly. Each client instance is its own isolated identity:

- **C++ client** → one account, keys in `~/.securemsg/`
- **Browser client** → one account, keys in that browser's IndexedDB

They're completely separate accounts — different usernames, different keypairs registered on the server. You can't log into the same account from both, and you wouldn't want to for the demo anyway.

For your **two-user demo** this actually works perfectly:

- Alice on the C++ client
- Bob on the browser (or vice versa)
- They exchange messages end-to-end encrypted via X3DH + Double Ratchet

That's a cleaner story than trying to show the same account on two devices. It also demonstrates **cross-client interoperability** which is actually impressive — the crypto protocol works regardless of whether the client is C++ or a browser.

what if bob wants to log on in edge and also chrome

Same problem — IndexedDB is **per browser profile**. Edge and Chrome have completely separate IndexedDB stores, so Bob's private keys in Edge don't exist in Chrome.

Bob would have to register a second account in Chrome. Same constraint as the C++ vs browser situation.

For your demo, this is a non-issue — just pick one browser and stick to it. Not worth the headache of trying to work around it.

okay - I think im going to send an email to my professor Eoin clarifying this and making sure he doesnt want anything more - can you draft this up? explain what we have done - what we dont have and why and ask if its smthn that would improve the our grade

Searched memory >

Okay, I've got the full picture of what Eoin's rubric expects. Let me draft this — the key thing to address is that you implemented X3DH + Double Ratchet (which is arguably *stronger* than HPKE Mode_Auth) rather than the literal HPKE spec, and you want to know if that's fine or if it'll cost you marks.

Subject: EPIC Project — Cryptography Implementation Clarification

Hi Eoin,

I hope you're well. I wanted to reach out ahead of Wednesday's submission to clarify our cryptographic implementation and check whether there's anything we should address before the deadline.

For key establishment and sender authentication, rather than implementing HPKE Mode_Auth (RFC 9180) as described in the guide, our team implemented the Signal Protocol — specifically X3DH (Extended Triple Diffie-Hellman) for initial key establishment and the Double Ratchet Algorithm for per-message key evolution. Both are built on X25519 and HKDF-SHA256, and message encryption uses AES-256-GCM throughout.

Our reasoning was that X3DH + Double Ratchet provides a strictly stronger set of properties than HPKE Mode_Auth: sender authentication is achieved via the identity key binding in X3DH, and we additionally gain forward secrecy and post-compromise security from the Double Ratchet, which HPKE alone does not provide. The rubric criterion reads "HPKE Mode_Auth (or equivalent justified construction)" — we believe our implementation qualifies as an equivalent justified construction, and we intend to defend that choice in the design document and at interview.

The remaining components are in line with the rubric: Argon2id password storage, HKDF for key



Send via Gmail



There's the draft. A couple of things worth noting about the approach taken:

The rubric literally says "**HPKE Mode_Auth (or equivalent justified construction)**" — X3DH + Double Ratchet is a very strong argument for that clause, since it gives everything HPKE gives *plus* forward secrecy and post-compromise security. Framing it as a superset rather than a deviation is the right move.

The multi-device point is included because it pre-empts any question he might have about cross-device access, and framing it as a conscious design decision (rather than something you missed) is better going into the interview.

okay cut that down a lot - get rid of the big justification for X3DH and DR because he himself said we should implement it for an A1 - just explain what we have - the area i wanted to flag bit is good - but just double check are we using sodium_malloc at all? as I thought it was being handled by python no? all im looking for from this email is clarity that one account per device is okay

Good catch — let me check the sodium_malloc usage.

Searched project for “sodium_malloc private key memory C++” >

`sodium_malloc` is actually used in the C++ client's `User.cpp` — it holds the identity keypair in locked memory at runtime. But you're right that the actual X3DH/Double Ratchet crypto (the session keys, ratchet state etc.) all lives in the Python daemon. So both are true — `sodium_malloc` in C++, Python handles the Signal Protocol.

Either way, it's not really relevant for this email. Here's the trimmed version:

Subject: EPIC Project — Quick Clarification

Hi Eoin,

Just a quick one ahead of Wednesday. Our implementation stores private keys locally and device-specifically — non-extractable CryptoKey objects in IndexedDB on the browser client, and encrypted key files on disk for the C++ client. As a result, a single account can't be accessed from multiple devices or browsers simultaneously.

We plan to flag this as a known limitation in the design document, but wanted to check — is this something you'd expect us to address for full marks, or is one account per device acceptable given the scope of the project?

Thanks,
Matthew



Send via Gmail



Nice and short. It explains the technical reason without over-justifying, acknowledges it in the design doc, and asks exactly what you need to know.

okay shorter but still mentioned we are using signal or 3xdh and dr or whatever - and i thought the problem wasnt simulatenuos access , its that once an account is created it cant be functional on another device as private keys are stored locally , no?

You're right — it's not about simultaneous access, it's that the account is fundamentally tied to the device it was created on. Keys can't move, so logging in from another device would give you an account shell with no crypto capability.

Subject: EPIC Project — Quick Clarification

Hi Eoin,

Quick one ahead of Wednesday. We implemented X3DH and the Double Ratchet for key establishment and messaging. As part of this, private keys are stored locally and never leave the device — so an account created on one device can't be used on another, as there'd be no key material to perform any crypto operations.

We're planning to note this as a known limitation in the design document, but wanted to check if it's something you'd expect us to address for full marks.

Thanks,
Matthew



Send via Gmail



That's the one. Mentions X3DH and Double Ratchet, correctly frames it as account-tied-to-device rather than a simultaneous access issue, and asks exactly what you need.

i think i say this;

Hi Eoin,

Quick one ahead of Wednesday. We implemented X3DH and the Double Ratchet for key establishment and messaging. As part of this, private keys are stored locally and never leave the device — so an account created on one device can't be used on another, as there'd be no key material to perform any crypto operations.

We're planning to note this as a known limitation in the design document, but wanted to check if it's something you'd expect us to implement for full marks.

Thanks,
Matthew

Looks good to me — "implement" rather than "address" is the right word there too, since you're asking whether he expects you to build device linking, not just document around it. Send it.

he responded:

What you've got here is a very strong, maximally conservative security property. Private keys never leave the device (and are presumably encrypted at rest on the device). Sure, there's a usability limitation in that accounts aren't portable, but that's a defensible position for the interview. You can always suggest (but not implement) a solution in that

Show more

That's a great response — he's essentially told you it's a strength, not a weakness, and given you the exact framing to use in the design doc. Key things to note for when you're writing it up:

- **Lead with it as a feature:** private keys never leave the device, server admin gets nothing usable
- **The trade-off to acknowledge:** account portability — moving to a new device would require an out-of-band key bundle transfer
- **Why server-side escrow would be worse:** password-derived keys have ~40-50 bits of entropy, making encrypted escrow attackable; you'd need HSMs/secure enclaves (what Signal does) to do it properly
- **Don't implement anything new** — he's explicitly said not to bother with device portability given the deadline

Solid reply to get from him three days out.

excerpt_from_previous_claude_message.txt

1 line

TXT

what is escrow?

Escrow just means a third party holding something on your behalf — in this context, storing your encrypted private keys on the server so you could download them from any device.

The problem Eoin was pointing out is that if the server holds your encrypted private key, an attacker who compromises the server can just keep trying passwords against it offline. A random 256-bit encryption key is unguessable, but a human-chosen password only has maybe 40-50 bits of effective entropy — which is potentially crackable given enough compute and time.

So the attack is: steal the encrypted key blob from the server, then brute-force the password offline at your leisure. That's the threat that server-side escrow introduces.

what is libsignal and do we not use it? what library do we use

libsignal is Signal's official open-source cryptographic library — it implements X3DH, Double Ratchet, and all the associated primitives in a production-hardened way. It's what the actual Signal app uses under the hood.

You don't use it. You implemented X3DH and Double Ratchet **from scratch** in Python using the `cryptography` library (hazmat primitives — X25519, HKDF, AES-GCM etc.). That's actually a stronger position for the interview and design doc, because you can explain every component yourself rather than just saying "we used libsignal." Eoin is assessing whether you understand the crypto, and rolling your own implementation (on top of vetted primitives, not truly from scratch) demonstrates that far better.

no he wants us to use librarys that are trusted cus we arent supposed to roll our own crypto

You're right, and there's a distinction worth being clear on:

You didn't roll your own crypto — you used vetted primitives from the `cryptography` library (Python's most trusted crypto library, backed by OpenSSL/Rust). What you built yourself is the **protocol logic** — the X3DH handshake flow and Double Ratchet state machine — on top of those primitives. That's totally legitimate and is actually what most Signal Protocol implementations do.

Rolling your own crypto would mean implementing AES-GCM yourself, or writing your own X25519 curve math. You didn't do that — you called `X25519PrivateKey`, `AESGCM`, `HKDF` etc. from the `cryptography` library.

So the correct way to describe it in your design doc is: "**We implemented the Signal Protocol (X3DH + Double Ratchet) using the Python `cryptography` library for all underlying primitives.**" That's fully defensible.

what files is that in?

Searched project for “cryptography library imports X25519 AESGCM HKDF” >

The Signal Protocol logic (X3DH + Double Ratchet) lives across these files:

Python daemon (the main implementation):

- `crypto-daemon/double_ratchet.py` — Double Ratchet state machine, using `cryptography` library (`X25519PrivateKey` , `AESGCM` , `HKDF`)
- `crypto-daemon/handlers.py` — X3DH send/receive operations
- `crypto-daemon/session_store.py` — session persistence

Browser frontend:

- `frontend/crypto.js` — X3DH and Double Ratchet reimplemented using the **Web Crypto API** (`crypto.subtle`) — `X25519` , `HKDF-SHA256` , `AES-GCM`

C++ client:

- `client-cpp/src/User.cpp` — identity keypair only, using **libsodium** (`crypto_kx_keypair` , `crypto_sign_keypair`)

So the trusted libraries across the stack are: **PyCA** `cryptography` (Python), **Web Crypto API** (browser), and **libsodium** (C++). All three are explicitly named as approved in the rubric.

yeah how does web3.js implementation differ from the cryptography library implementation

Good question — they're not actually alternatives to each other, they serve completely different purposes:

Web Crypto API (`crypto.subtle`) is a browser-native API for cryptographic operations — encryption, key generation, signing etc. It's what your frontend uses to do X3DH and Double Ratchet in the browser.

PyCA `cryptography` is a Python library that does the same category of thing — encryption, key generation, signing etc. It's what your Python daemon uses.

They're just the same job in different environments — one is browser JavaScript, one is Python.

web3.js is something completely different — it's nothing to do with cryptography in the Signal sense. It's a JavaScript library for interacting with the Ethereum blockchain (sending transactions, calling smart contracts etc.). That's Julia's domain for the blockchain component, not related to your messaging crypto at all.

So to directly answer — web3.js and the `cryptography` library don't really "differ" in a meaningful way for your project because they're not doing the same thing. The comparison that makes sense is Web Crypto API vs `cryptography` library — and the answer there is just that they're the same primitives, different languages.

but yk the way we kind of assemble X3DH and Double ratchet ourselves -
do we do the saem with Web3.js

No — web3.js you just call it as-is. You're not assembling anything with it, you're just calling methods like `contract.methods.recordDigest(hash).send()` to interact with the smart contract on Sepolia. It's much more straightforward.

The reason you had to assemble X3DH and Double Ratchet yourself is that there's no library that just gives you `x3dh_send()` and `double_ratchet_encrypt()` as ready-made functions (that's essentially what libsignal does, but you didn't use it). So you had to wire together the individual primitives — X25519, HKDF, AES-GCM — in the correct sequence to implement the protocol.

web3.js already gives you exactly the high-level functions you need for blockchain interaction, so there's nothing to assemble.

so we did roll our own crypto..?:

OK, I've read the repo. Here's what you meant, stated plainly:

The current crypto-daemon is a hand-rolled Signal Protocol implementation. Your crypto-daemon/ has custom Python code doing X3DH key agreement, the Double Ratchet state machine, ratchet session serialisation, prekey management — all written from scratch.*don't do

[Show more](#)

Yeah, technically you did roll your own in the sense that matters — you implemented the protocol state machine yourself rather than using a library that hands you the full Signal Protocol. Using vetted primitives (X25519, HKDF, AES-GCM from the `cryptography` library) is the right call and what Eoin means by "don't roll your own crypto" — you didn't implement AES yourself or anything like that. But the protocol logic on top of those primitives is your own code.

The good news is the last paragraph of that response nails it — for a graded university project where Eoin is assessing whether you understand the crypto, your custom implementation is arguably **better** than dropping in libsignal as a black box. You can walk through every line of `double_ratchet.py` and explain exactly what it does and why. That's what the interview is testing.

So the honest answer is: yes you rolled the protocol, no you didn't roll the primitives, and that's actually fine for this context. Just don't describe it as "we used libsignal" — describe it as "we implemented X3DH and Double Ratchet using PyCA cryptography for all underlying primitives."

excerpt_from_previous_claude_message.txt

1 line

TXT

no. he wants us to use a trusted library NOT TO ROLL OUR OWN CRYPTO
- we need to change to something well known and secure

Fair enough — that's a clear instruction. The problem is the practical reality, which that response already laid out:

- **libsignal** has no official Python package on PyPI — you'd be compiling Rust from source, which is a nightmare three days before deadline
- **Community Python wrappers** exist but are poorly maintained

Realistically your options are:

1. **Leave it as-is and document it well** — acknowledge in the design doc that you implemented the protocol over vetted primitives (PyCA `cryptography`) and that libsignal has no stable Python binding, making it impractical for this stack
2. **Switch the daemon to Node.js** and use `libsignal`'s Node.js bindings, which are officially maintained by Signal — but that's a significant rewrite three days out
3. **Use the browser frontend as your "trusted library" story** — the Web Crypto API IS the trusted library for the browser side, that part is clean

Honestly given the deadline I'd raise this directly with Eoin — explain that you're aware libsignal exists, that there's no stable Python binding, and ask what he'd consider acceptable. His response about the design doc being sufficient suggests he's pragmatic. You already have his email open.

The messaging application must provide end-to-end encrypted (E2EE) communication between users. When Alice sends a message to Bob, only Bob c...

PASTED

Okay well lets look at the brief and the requirements and then we can email him for clarity - oen thing to note is:

3. The brief's exact wording on this is important. "Never roll your own" in the brief says:

Forbidden: hand-rolled primitives

The word is primitives. AES, SHA, HKDF — those are primitives. X3DH and Double Ratchet are protocols built from primitives. Your primitives are all

Show more

Reading the brief carefully, your implementation looks valid. Section 4a explicitly lists `cryptography` (PyCA) as an approved library — that's exactly what you're using. Section 4c forbids "hand-rolled primitives" and lists specific examples: MD5, SHA-1, DES, ECB, textbook RSA. X3DH and Double Ratchet are not primitives, they're protocol compositions built from approved primitives.

The brief's own example for key establishment is "e.g. HPKE Mode_Auth" — the word "e.g." means it's illustrative, not prescriptive. You've implemented something stronger.

So honestly you're probably fine without emailing. But if you want clarity, keep it short:

Subject: EPIC — Quick Clarification on Implementation Standards

Hi Eoin,

Quick clarification on section 4c of the brief. We've implemented X3DH and the Double Ratchet as a protocol layer on top of PyCA cryptography primitives (X25519, HKDF-SHA256, AES-256-GCM). We haven't hand-rolled any primitives — all underlying crypto comes from the cryptography library.

Does composing X3DH and Double Ratchet ourselves from approved primitives satisfy the implementation standards, or would you consider this hand-rolling in the sense the brief forbids?

Thanks,
Matthew



Send via Gmail



That's all you need to ask. His answer will either confirm you're fine, or tell you exactly what he wants changed — and you'd still have time to act on it before Wednesday.

it will be in resaponse to the following email so i shud ack that:

Hi Matthew,

I'm going to counter this one with a frame challenge: is that really a limitation?

What you've got here is a very strong, maximally conservative security property. Private keys never leave the device (and are presumably

Show more

Subject: Re: EPIC Project — Quick Clarification

Hi Eoin,

Thanks for the detailed response, that's really helpful framing for the design doc.

One further question while I have you — section 4c of the brief forbids hand-rolled primitives. We've implemented X3DH and the Double Ratchet as a protocol layer on top of PyCA cryptography primitives (X25519, HKDF-SHA256, AES-256-GCM) — we haven't implemented any primitives ourselves. Does composing the protocol from approved primitives satisfy the implementation standards, or would you consider this hand-rolling in the sense the brief forbids?

Thanks,
Matthew



Send via Gmail



There you go — acknowledges his previous reply briefly and moves straight to the new question.

maybe ask about using the libsignal library through node.js vs rust if we need to rewrite

Subject: Re: EPIC Project — Quick Clarification

Hi Eoin,

Thanks for the detailed response, that's really helpful framing for the design doc.

One further question while I have you — section 4c of the brief forbids hand-rolled primitives. We've implemented X3DH and the Double Ratchet as a protocol layer on top of PyCA cryptography primitives (X25519, HKDF-SHA256, AES-256-GCM) — we haven't implemented any primitives ourselves. Does composing the protocol from approved primitives satisfy the implementation standards, or would you consider this hand-rolling in the sense the brief forbids?

If a rewrite is necessary, would using libsignal via its Node.js bindings be acceptable, or would you expect the Rust library directly?

Thanks,
Matthew



Send via Gmail



That covers both angles — clears up the primitives question first, and only raises the rewrite option if he says it's needed.

and does the rewrite option make sense?

Honestly not really, for a few reasons:

- Your crypto daemon is Python, not Node.js — rewriting it to use libsignal's Node.js bindings would mean rewriting the entire daemon in a different language three days out
- The C++ client communicates with the daemon over TCP loopback — that interface would need to stay the same regardless
- libsignal's Node.js bindings are also not trivial to set up and aren't well documented

The more realistic rewrite path if Eoin insists would actually be to drop the Python daemon entirely and call libsignal's Node.js bindings from the **browser frontend** instead — but that conflicts with your architecture where the C++ client relies on the daemon.

Realistically if he says a rewrite is needed you'd be in trouble given the deadline. The better framing in the email is probably just asking whether PyCA primitives + hand-rolled protocol is acceptable, and leaving the rewrite question out — because if the answer is "yes it's fine" you've saved yourself the panic, and if the answer is "no" you'd want to have a proper conversation with him rather than commit to a specific rewrite path in an email.

Want me to drop the rewrite line?

yeah like how could we implement signal in the cpp client without using python cus idk using a node js daemon would that work?

The C++ client could use libsignal directly via its **Rust FFI** — libsignal exposes a C API that C++ can call through FFI. But that's a significant undertaking even for experienced developers, let alone three days before a deadline.

The Node.js daemon idea doesn't really make sense either — you'd just be swapping one daemon language for another, and Node.js talking to a C++ client over TCP loopback is no cleaner than Python doing the same thing. You'd still have the same architecture, just with a rewrite cost.

Honestly the cleanest path if Eoin said a change was needed would be to keep the Python daemon but swap out your hand-rolled protocol for a community Python wrapper around libsignal — but as mentioned earlier those wrappers are poorly maintained and there's no official one on PyPI.

The bottom line is: **there's no clean way to drop in libsignal given your current stack without a significant rewrite.** Which is exactly why the email should just ask whether what you have is acceptable — because the practical alternative is genuinely painful. Drop the rewrite question from the email and let him answer the main question first.

i think cus node.js can access libsignal no?

Yes, Node.js has official libsignal bindings maintained by Signal themselves — it's probably the most accessible way to use libsignal outside of Java/Swift.

But the issue is your daemon is currently Python talking to your C++ client over TCP loopback with newline-delimited JSON. You *could* rewrite the daemon in Node.js and keep the exact same TCP loopback JSON interface — the C++ client wouldn't care what language is on the other end as long as the protocol is the same.

So theoretically: **yes, a Node.js daemon using libsignal is a viable architecture.**

Whether it's viable in 3 days is a different question — you'd be rewriting all of `double_ratchet.py`, `x3dh.py`, `handlers.py`, `session_store.py` etc. in Node.js using a library you haven't used before.

Still think you should drop the rewrite question from the email and just ask whether your current implementation is acceptable. No point opening that can of worms unless he tells you it's necessary.

nah i think ill leave it as thats our best back up option

Fair enough, makes sense to have it in there. Send it as is then.

Hi Matthew,

That's a thorny question! It's not strictly forbidden; I'm not going to go after it in the same way that I'd persecute hand-rolled AES or HKDF.

That being said, it does bring more material under scrutiny. Protocol implementations aren't trivial, so expect me to have read your implementation and prepared questions for the interview. If you've done

Show more

Okay this is actually a really useful response. He's not penalising you for it but he's telling you exactly what he'll be testing. Let's go through his list against what you know about your implementation:

Things he'll probe:

1. **SPK signature verification** — this was literally your last blocker ("SPK signature verification failed"). You need to make sure this is actually working correctly, not just bypassed
2. **DoS handling** — $N=4,000,000,000$ chain key derivation. Is there any cap on skipped message keys in your Double Ratchet?
3. **Authentication failures on tampered messages** — does your code correctly reject and handle AES-GCM auth tag failures?
4. **Nonce handling** — are nonces ever reused? How are they generated?
5. **Ratchet state at rest** — your encrypted session files in `~/.securemsg/sessions/`
6. **IK dual use for X25519 and Ed25519** — using the same identity key for both DH and signing is a known footgun

The SPK one is the most urgent given where you left off. Do you know whether the verification failure was actually fixed or just worked around?

Okay i want to send a big prompt to claude code asking it to test / check for all those things and make sure we have a clean implementation - give me the prompt - im thinking of using opus for it with high effort

Let me read the actual implementation files first before writing the prompt, so it's accurate and specific rather than generic.

Searched memory >

Good — I have a solid picture of the implementation. Here's a thorough Claude Code prompt:

Prompt for Claude Code (Opus, high effort):

Read the following files in full before making any changes:

- crypto-daemon/x3dh.py
- crypto-daemon/double_ratchet.py
- crypto-daemon/handlers.py
- crypto-daemon/session_store.py
- frontend/crypto.js

Eoin O'Brien (our cryptography examiner) has told us he will specifically probe the following areas at interview. Audit the entire implementation against each point, fix any gaps, and explain what you found and changed:

1. SPK SIGNATURE VERIFICATION

Confirm that Ed25519 SPK signature verification is happening correctly in both x3dh.py (Python daemon) and frontend/crypto.js (browser) BEFORE any DH operations proceed. Verify that a tampered or missing signature causes a hard abort with a clear error, not a silent failure or bypass.

2. DOS PROTECTION – SKIPPED MESSAGE KEYS

In double_ratchet.py, MAX_SKIPPED is currently 100. Confirm that this cap is enforced correctly in ALL code paths that call advance_chain() in a loop (including _dh_ratchet_recv). A malicious message with n=4000000000 must never cause the loop to run billions of iterations – it must be rejected before the loop begins. Add an explicit pre-check: if (n - self.n_recv) > MAX_SKIPPED: raise ValueError before the while loop. Do the same audit in frontend/crypto.js – confirm _evictSkippedIfFull is called and check whether the same pre-check exists there.

3. AEAD AUTHENTICATION FAILURE HANDLING

In double_ratchet.py _open(), confirm that AESGCM.decrypt() failure raises an exception that propagates correctly up through decrypt() and op_decrypt_message() in handlers.py all the way back to the client as an error response – never returning partial plaintext or silently swallowing the

exception. Check the same in frontend/crypto.js – confirm AES-GCM decryption failure throws and is not caught and suppressed anywhere.

4. NONCE HANDLING

In double_ratchet.py encrypt(), confirm nonces are generated with secrets.token_bytes(12) (CSPRNG, never hardcoded, never reused). Confirm nonces are included in the returned message dict and sent to the peer. Confirm nonces are never reused across messages (each encrypt call generates a fresh nonce). Do the same audit in frontend/crypto.js.

5. RATCHET STATE AT REST

In session_store.py, confirm that ratchet sessions are encrypted at rest with AES-256-GCM under the wrap_key derived from the user's passphrase. Confirm the nonce for this encryption is freshly generated on every save (never hardcoded or reused). Confirm that loading a session with a wrong wrap_key raises an exception rather than returning corrupt state.

6. IK DUAL USE – X25519 AND Ed25519

Confirm that the identity key is correctly split into TWO separate keypairs:

- An X25519 keypair (ik_priv / ik_pub) used only for DH operations in X3DH
- An Ed25519 keypair (sign_priv / sign_pub) used only for signing the SPK

Confirm that the X25519 IK private key is NEVER used for signing, and the Ed25519 signing key is NEVER used for DH. Check this in identity.py, handlers.py, x3dh.py, and frontend/crypto.js.

For each of the 6 points above:

- State what you found (correct / incorrect / missing)
- Make any necessary fixes
- Write a brief explanation of what the fix does and why it matters

Do not refactor unrelated code. Do not change the TCP IPC interface or JSON protocol. Do not change the session file format unless strictly necessary for point 5.

That should cover everything Eoin flagged. The DoS pre-check (point 2) is the one most likely to be missing — the `MAX_SKIPPED` cap exists but I'm not confident there's an explicit guard before the while loop runs at all.

excerpt_from_previous_claude_message.txt

1 line

TXT

do i want any changes or just a diagnosis?

Good point — you probably want a diagnosis first so you know exactly what's broken before anything gets touched, especially this close to the deadline.

Change the opening to:

Read the following files in full. Do NOT make any changes to any file.

For each of the 6 points below, audit the implementation and report:

- What you found (correct / gap / missing)
- The exact file, function, and line number of any issue
- What a fix would look like (describe it, don't apply it)

Wait for my confirmation before touching anything.

Then once you see the report you can decide what's actually worth fixing vs just acknowledging in the design doc at this stage.

I've read all six files in full. Here's my audit. Per your instruction, I have not changed anything — findings and proposed fixes only. One genuine vulnerability...

PASTED

Great audit. The JS DoS is the real one to fix — that's a genuine vulnerability Eoin would catch. The Python one is more about satisfying his exact wording.

Both fixes are small and surgical — just adding pre-checks before two while loops each. I'd say go ahead and apply both. Want to send the go-ahead to Claude Code?

Both files pass syntax checks. Here's what I changed and why it matters.

Fix 1 — `frontend/crypto.js` (the real DoS hole)

Added a `MAX_SKIPPED = 100` constant and an explicit pre-check before both skip loops in `ratchetDecrypt`:

- Before the old-chain drain loop (`[crypto.js:447]`)(`vscode-webview://1t5i1480760aconv518l0ab9gmj9psjatpvkqbqse38u1audf6de`

Show more

Clean. Both fixes are exactly right and the false-positive verification is exactly what Eoin will want to hear — that you checked the pre-checks don't fire on legitimate ratchet steps.

Make sure you can articulate point 5 (the silent-skip at rest) in the interview — frame it as a deliberate design choice: raising on a bad wrap_key during `load_all_sessions` would block all sessions from loading if one file is corrupt, so skipping is correct behaviour, and the security property still holds because AES-GCM authentication prevents corrupt plaintext ever being returned.

Commit those changes and you're in good shape on the crypto audit.

give me the cmds to cmmit these changes

```
git add crypto-daemon/double_ratchet.py frontend/crypto.js
git commit -m "fix: add MAX_SKIPPED pre-checks before skip loops in DR and frontend"
git push
```

to what branch

What branch are you currently on? Run `git branch` and let me know — given you're in late-stage development it's probably `main` but worth checking before you push.

```
PS C:\Users\m4tth\EPIC> git branch
feature/cpp-client-implementation
feature/message-persistence
* feature/revoke-access
feature/websockets
fix/x3dh-key-exchange
main
PS C:\Users\m4tth\EPIC>
```

You're on `feature/revoke-access` — that's probably not where you want these crypto fixes. Given the deadline I'd push straight to main:

```
git add crypto-daemon/double_ratchet.py frontend/crypto.js
git commit -m "fix: add MAX_SKIPPED pre-checks before skip loops in DR and
frontend"
git checkout main
git merge feature/revoke-access
git push origin main
```

Though — what's actually on `feature/revoke-access`? Is that branch up to date with your recent work or did you end up on it by accident?

idk but everything has been pushed and stuff i believe - i just need to make a new branch for this

```
git checkout -b fix/dos-skipped-keys
git add crypto-daemon/double_ratchet.py frontend/crypto.js
git commit -m "fix: add MAX_SKIPPED pre-checks before skip loops in DR and
frontend"
git push origin fix/dos-skipped-keys
```

Then merge to main when you're happy with it.

